📝 Exámenes de práctica A · B · C (para hacer)

Examen A

📝 Examen de práctica A · para hacer

10 preguntas, una por sección, al estilo del modelo. Escribe tu código en las celdas vacías. Compara con la Salida esperada. En las de **lectura** (1 y 10), escribe tu predicción antes de comprobar.

Sección 1 · El equipo compartido

Lee el código y di qué imprime. Explica por qué.

```
class Equipo:
    jugadores = []
    def __init__(self, nombre):
        self.nombre = nombre
        self.jugadores.append(nombre)

e1 = Equipo("Rojo")
e2 = Equipo("Azul")
print(len(Equipo.jugadores), e1.jugadores, e2.jugadores)
```

✍️ **Mi predicción / respuesta:**

Sección 2 · Notas del curso

Tienes `notas = [3, 5, 8, 10, 4, 7, 6]`. Con **una list comprehension** crea una lista que ponga `"APTO"` si la nota es 5 o más y `"NO APTO"` si no. Después muestra cuántos están aptos.

Salida esperada:

```
['NO APTO', 'APTO', 'APTO', 'APTO', 'NO APTO', 'APTO', 'APTO']
Aptos: 5
```

Sección 3 · Tienda con map, filter y reduce

Trabaja con esta lista:

```
productos = [
    {"nombre": "Teclado", "precio": 20.0, "oferta": True},
    {"nombre": "Ratón",   "precio": 12.0, "oferta": False},
    {"nombre": "Monitor", "precio": 150.0, "oferta": True},
    {"nombre": "Cable",   "precio": 5.0,  "oferta": False},
]
```

1. Con `map()`, una lista con los nombres en mayúsculas.
2. Con `filter()`, los nombres de los productos en oferta.
3. Con `reduce()`, el precio total de todos los productos.

Salida esperada:

```
['TECLADO', 'RATÓN', 'MONITOR', 'CABLE']
['Teclado', 'Monitor']
187.0
```

Sección 4 · Potencia recursiva

Escribe una función recursiva `potencia(base, exp)` que calcule `base` elevado a `exp` **sin usar `**` ni bucles**. Caso base: exponente 0 devuelve 1.

```
print(potencia(2, 5))
print(potencia(10, 0))
```

Salida esperada:

```
32
1
```

Sección 5 · Clase Pelicula

Crea una clase `Pelicula` con: - atributos `titulo`, `duracion` (minutos) y `vista` (por defecto `False`); - método `marcar_vista()` que ponga `vista` a `True`; - método `es_larga()` que devuelva `True` si dura más de 120 minutos; - `__str__` con el formato `Origen (148 min) [pendiente]` o `[vista]` según corresponda.

```
p = Pelicula("Origen", 148)
print(p)
print(p.es_larga())
p.marcar_vista()
print(p)
```

Salida esperada:

```
Origen (148 min) [pendiente]
True
Origen (148 min) [vista]
```

Sección 6 · Clase Temperatura

Crea una clase `Temperatura` que guarde `grados`. Haz que funcionen: - `+` entre dos temperaturas → un **nuevo** `Temperatura`; - `<` que compare los grados; - `str()` con el formato `21.5 °C` (1 decimal); - `float()` que devuelva los grados.

```
a = Temperatura(21.5)
b = Temperatura(3.2)
print(a + b)
print(a < b)
print(float(a))
```

Salida esperada:

```
24.7 °C
False
21.5
```

Sección 7 · Empleados

1. Crea `Empleado` con `nombre` y `sueldo_base`, y un método `sueldo()` que devuelva el sueldo base.
2. `Comercial` (hereda de `Empleado`): recibe además `ventas` y suma 0.10 € por cada venta al sueldo base (usa `super()`).
3. `Jefe` (hereda de `Empleado`): cobra el sueldo base más un 20 %.
4. Recorre una lista con los tres tipos mostrando `nombre: sueldo` (2 decimales) y el total.

```
plantilla = [
    Empleado("Ana", 1000),
    Comercial("Luis", 1000, 50),
    Jefe("Marta", 1000),
]
```

Salida esperada:

```
Ana: 1000.00
Luis: 1005.00
Marta: 1200.00
Total: 3205.00
```

Sección 8 · Figuras geométricas

Este código usa `if/elif` por tipo:

```python
figuras = [["cuadrado", 4], ["circulo", 3], ["cuadrado", 2]]
for f in figuras:
    if f[0] == "cuadrado":
        area = f[1] * f[1]
    elif f[0] == "circulo":
        area = 3.14 * f[1] * f[1]
    print(f[0], round(area, 2))
```

Reescríbelo con una clase `Figura` y subclases `Cuadrado` y `Circulo`, cada una con un método `area()`, y recórrelo **sin ningún `if` de tipo**. El `lado`/`radio` se pasa al crear el objeto.

```python
figuras = [Cuadrado(4), Circulo(3), Cuadrado(2)]
for f in figuras:
    print(type(f).__name__, round(f.area(), 2))
```

Salida esperada:

```
Cuadrado 16
Circulo 28.26
Cuadrado 4
```

Sección 9 · Biblioteca

Clase `Libro` con `titulo`, `paginas` y `precio`. Con esta lista:

```python
libros = [
    Libro("Python", 300, 25.0),
    Libro("Historia", 500, 30.0),
    Libro("Cuento", 80, 10.0),
]
```

1. Con una comprehension, lista de títulos de libros de más de 100 páginas.
2. Con `map()` y `reduce()` (o `sum`), el precio total.
3. Función recursiva `paginas_totales(libros)` que sume las páginas **sin bucles**.

Salida esperada:

```
['Python', 'Historia']
65.0
880
```

Sección 10 · El módulo saludos

Lee este código y di qué imprime y por qué.

Archivo `saludos.py`:

```python
print("Cargando saludos")
IDIOMA = "ES"
def hola(nombre):
    return "Hola " + nombre
if __name__ == "__main__":
    print(hola("test"))
```

En otro archivo:

```
import saludos
import saludos
from saludos import hola
print(hola("Ana"))
```

¿Cuántas veces se imprime `"Cargando saludos"`? ¿Se imprime `"Hola test"`? ¿Por qué?

✍️ **Mi predicción / respuesta:**

Examen B

📝 Examen de práctica B · para hacer

10 preguntas, una por sección, al estilo del modelo. Escribe tu código en las celdas vacías. Compara con la Salida esperada. En las de **lectura** (1 y 10), escribe tu predicción antes de comprobar.

Sección 1 · La mochila por defecto

Lee el código y di qué imprime. Explica por qué.

```
def anadir(objeto, mochila=[]):
    mochila.append(objeto)
    return mochila

print(anadir("libro"))
print(anadir("boli"))
print(anadir("agua", ["mapa"]))
```

✍️ **Mi predicción / respuesta:**

Sección 2 · Palabras largas

Tienes `palabras = ["sol", "ordenador", "mar", "python", "ia", "programa"]`. Con **una set comprehension** obtén las palabras de más de 3 letras en MAYÚSCULAS, sin repetir. Muéstralas ordenadas y cuántas son.

Salida esperada:

```
['ORDENADOR', 'PROGRAMA', 'PYTHON']
Total: 3
```

Sección 3 · Empleados funcional

Trabaja con:

```
empleados = [
    {"nombre": "Ana",   "salario": 1800, "activo": True},
    {"nombre": "Luis",  "salario": 1200, "activo": False},
    {"nombre": "Marta", "salario": 2500, "activo": True},
    {"nombre": "Hugo",  "salario": 900,  "activo": True},
]
```

1. Con `sorted()` y una lambda, los nombres ordenados de **mayor a menor** salario.
2. Con `filter()`, los nombres de los empleados activos.
3. Con `reduce()`, el salario más alto de todos.

Salida esperada:

```
['Marta', 'Ana', 'Luis', 'Hugo']
['Ana', 'Marta', 'Hugo']
2500
```

Sección 4 · Contar vocales

Escribe una función recursiva `contar_vocales(texto)` que devuelva cuántas vocales hay, **sin bucles**. Pista: mira el primer carácter y suma 1 si es vocal, luego sigue con el resto.

```
print(contar_vocales("murcielago"))
print(contar_vocales("xyz"))
```

Salida esperada:

```
5
0
```

Sección 5 · Clase CuentaPuntos

Crea una clase `CuentaPuntos` con: - atributo de clase `total_jugadores` que cuente cuántas se crean; - atributos `jugador` y `_puntos` (0 por defecto); - método `sumar(p)` que sume puntos solo si `p` es positivo; - método `__str__` con el formato `Ana: 30 puntos`.

```
c1 = CuentaPuntos("Ana")
c2 = CuentaPuntos("Leo")
c1.sumar(30)
c1.sumar(-5)
print(c1)
print(c2)
print("Jugadores:", CuentaPuntos.total_jugadores)
```

Salida esperada:

```
Ana: 30 puntos
Leo: 0 puntos
Jugadores: 2
```

Sección 6 · Clase Cesta

Crea una clase `Cesta` con un `nombre` y una lista interna de productos. Haz que funcionen: - `len(cesta)` → número de productos; - `"X" in cesta` → si un producto está; - `cesta += "X"` → añade y devuelve la propia cesta; - `str(cesta)` → `Compra (2 productos)`.

```
c = Cesta("Compra")
c += "pan"
c += "leche"
print(c)
print(len(c))
print("pan" in c)
print("huevos" in c)
```

Salida esperada:

```
Compra (2 productos)
2
True
False
```

Sección 7 · Animales (polimorfismo)

1. Crea `Animal` con `nombre` y un método `sonido()` que devuelva `"..."`.
2. `Perro` devuelve `"Guau"`, `Gato` devuelve `"Miau"`.
3. `Cachorro` hereda de `Perro` y su `sonido()` es el del padre más `" (flojito)"` (usa `super()`).
4. Recorre una lista de animales mostrando `nombre: sonido`.

```
animales = [Perro("Rex"), Gato("Tom"), Cachorro("Boti")]
```

Salida esperada:

```
Rex: Guau
Tom: Miau
Boti: Guau (flojito)
```

Sección 8 · Transporte (refactor)

Este código usa `if/elif` por tipo:

```
viajes = [["taxi", 10], ["bus", 10], ["metro", 10]]
for v in viajes:
    if v[0] == "taxi":
        precio = v[1] * 1.5 + 2
    elif v[0] == "bus":
        precio = v[1] * 0.5
    elif v[0] == "metro":
        precio = v[1] * 0.4
    print(v[0], round(precio, 2))
```

Reescríbelo con una clase `Viaje` y subclases `Taxi`, `Bus` y `Metro`, cada una con `precio()`. Los km se pasan al crear el objeto. Sin `if` de tipo.

```
viajes = [Taxi(10), Bus(10), Metro(10)]
for v in viajes:
    print(type(v).__name__, round(v.precio(), 2))
```

Salida esperada:

```
Taxi 17.0
Bus 5.0
Metro 4.0
```

Sección 9 · Gimnasio

Clase `Socio` con `nombre`, `edad` y `cuota`. Con:

```
socios = [
    Socio("Ana", 30, 40.0),
    Socio("Luis", 17, 25.0),
    Socio("Marta", 45, 50.0),
    Socio("Hugo", 16, 25.0),
]
```

1. Con un **set** comprehension, las edades distintas (ordénalas al mostrarlas).
2. Con `filter()`, los nombres de los socios mayores de edad (18 o más).
3. Función recursiva `ingresos(socios)` que sume las cuotas **sin bucles**.

Salida esperada:

```
[16, 17, 30, 45]
['Ana', 'Marta']
140.0
```

Sección 10 · Importaciones

Responde en papel (teoría de módulos): 1. ¿Qué diferencia hay entre `import utilidades` y `from utilidades import limpiar`? 2. En un paquete, ¿qué significa el punto en `from .config import IVA`? 3. ¿Para qué sirve el archivo `__init__.py`? 4. ¿Cuándo se ejecuta el bloque `if __name__ == "__main__":`? 5. Si importo el mismo módulo dos veces, ¿se ejecuta su código dos veces?

✍️ **Mi predicción / respuesta:**

Examen C (nivel alto)

📝 Examen de práctica C (nivel alto) · para hacer

El más difícil de los tres: MRO, reduce avanzado, recursión anidada. Una pregunta por sección.

Sección 1 · El closure contador

Lee el código y di qué imprime. Explica por qué.

```
def crear_contador():
    total = 0
    def sumar(n):
        nonlocal total
        total += n
        return total
    return sumar

c1 = crear_contador()
c2 = crear_contador()
print(c1(5), c1(3), c2(10), c1(1))
```

✍️ **Mi predicción / respuesta:**

Sección 2 · Matriz a diccionario

Tienes una matriz (lista de listas):

```
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

1. Con una comprehension de **dos** `for`, una lista plana solo con los números **pares**.
2. Con una dict comprehension, un diccionario `{numero: su cuadrado}` solo de los números de la **diagonal** (posiciones [0][0], [1][1], [2][2]). Pista: `matriz[i][i]`.

Salida esperada:

```
[2, 4, 6, 8]
{1: 1, 5: 25, 9: 81}
```

Sección 3 · Ventas con reduce avanzado

```
ventas = [
    {"vendedor": "Ana",  "producto": "A", "importe": 100, "devuelto": False},
    {"vendedor": "Luis", "producto": "B", "importe": 200, "devuelto": True},
    {"vendedor": "Ana",  "producto": "C", "importe": 150, "devuelto": False},
    {"vendedor": "Luis", "producto": "A", "importe": 300, "devuelto": False},
]
```

1. Con `filter()` y `reduce()`, el importe total de las ventas **no devueltas**.
2. Con `map()`, una lista de textos `"Ana vendió A por 100€"`.
3. Con `reduce()`, construye un diccionario `{vendedor: importe acumulado}` contando **solo las no devueltas**. Pista: el acumulador es un diccionario y empieza en `{}`.

Salida esperada:

```
550
['Ana vendió A por 100€', 'Luis vendió B por 200€', 'Ana vendió C por 150€', 'Luis vendió A por 300€']
{'Ana': 250, 'Luis': 300}
```

Sección 4 · Aplanar listas anidadas (recursivo)

Escribe una función recursiva `aplanar(lista)` que convierta una lista con listas dentro en una lista plana. Puedes usar un `for` para recorrer, pero las sublistas se tratan con la llamada recursiva. Pista: `isinstance(x, list)`.

```
print(aplanar([1, [2, 3], [[4], 5], [], 6]))
print(suma_profunda([1, [2, [3, [4]]]]))
```

Y una `suma_profunda(lista)` que sume todos los números por muy anidados que estén.

Salida esperada:

```
[1, 2, 3, 4, 5, 6]
10
```

Sección 5 · Clase Inventario (composición + classmethod)

Crea una clase `Producto` con `nombre`, `precio` y `stock`. Luego una clase `Inventario` que **contenga** una lista de productos con: - `añadir(producto)`; - `valor_total()` → suma de `precio × stock` de todos; - `sin_stock()` → lista de nombres con stock 0; - método de clase `desde_pares(pares)` que reciba una lista de tuplas `(nombre, precio, stock)` y devuelva un `Inventario` ya montado (usa `@classmethod`).

```
inv = Inventario.desde_pares([("Pan", 1.0, 10), ("Leche", 0.8, 0), ("Huevos", 2.0, 5)])
print(inv.valor_total())
print(inv.sin_stock())
```

Salida esperada:

```
20.0
['Leche']
```

Sección 6 · Clase Vector (operadores)

Crea una clase `Vector` con `x` e `y`. Haz que funcionen: - `+` → nuevo `Vector` sumando componente a componente; - `*` por un número → nuevo `Vector` con cada componente multiplicada; - `==` → compara ambas componentes; - `len(v)` → el módulo redondeado a entero (`int((x**2 + y**2) ** 0.5)`); - `str(v)` → `(3, 4)`.

```
a = Vector(3, 4)
b = Vector(1, 2)
print(a + b)
print(a * 2)
print(a == Vector(3, 4))
print(len(a))
```

Salida esperada:

```
(4, 6)
(6, 8)
True
5
```

Sección 7 · Herencia múltiple y MRO

Lee y predice la salida, y luego complétalo si hace falta.

```python
class Dispositivo:
    def describir(self):
        return ["dispositivo"]

class Pantalla(Dispositivo):
    def describir(self):
        return super().describir() + ["pantalla"]

class Bateria(Dispositivo):
    def describir(self):
        return super().describir() + ["batería"]

class Movil(Pantalla, Bateria):
    def describir(self):
        return super().describir() + ["móvil"]

print(Movil().describir())
print([c.__name__ for c in Movil.__mro__])
```

Pregunta: ¿por qué `"dispositivo"` aparece una sola vez si `Pantalla` y `Bateria` heredan las dos de `Dispositivo`? ¿A qué clase llama el `super()` de `Pantalla` dentro de un `Movil`?

✍️ **Mi predicción / respuesta:**

Sección 8 · Suscripciones (refactor + ampliación)

Refactoriza con clases y deja preparado para ampliar. Superclase `Plan` con `cliente` y `base`, método `coste()` que devuelve `base`, y `__str__` → `Básico: Ana paga 10.00 €` (el tipo con `type(self).__name__`). Subclases: - `Premium`: cuesta el doble de la base; - `Pro`: base + 5 € fijos. Crea una clase `Cartera` que contenga los planes, con `total()` (suma de costes) y `__str__` que liste todos. Luego **sin tocar nada**, añade un plan `Gratis` que cueste 0.

```python
cartera = Cartera()
cartera.añadir(Plan("Ana", 10))
cartera.añadir(Premium("Luis", 10))
cartera.añadir(Pro("Marta", 10))
cartera.añadir(Gratis("Hugo", 10))
print(cartera)
print(f"Total: {cartera.total():.2f} €")
```

Salida esperada:

```
Plan: Ana paga 10.00 €
Premium: Luis paga 20.00 €
Pro: Marta paga 15.00 €
Gratis: Hugo paga 0.00 €
Total: 45.00 €
```

Sección 9 · Pedidos de una cafetería (integrador)

```python
class Bebida:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
    def precio_final(self):
        return self.precio

class ConLeche(Bebida):      # +0.50 €
    def precio_final(self):
        return super().precio_final() + 0.50

class Grande(Bebida):        # +30 % sobre el precio
    def precio_final(self):
        return super().precio_final() * 1.30
```

Con esta lista:

```
pedidos = [
    {"bebida": ConLeche("Café", 1.20), "uds": 2},
    {"bebida": Bebida("Agua", 1.00),   "uds": 1},
    {"bebida": Grande("Zumo", 2.00),   "uds": 3},
]
```

1. Con una comprehension, lista con el importe de cada pedido (`precio_final() × uds`) redondeado a 2.
2. Con `map()` + `reduce()`, la facturación total.
3. Con `filter()` e `isinstance()`, los nombres de las bebidas `ConLeche`.
4. Recursiva `total_uds(pedidos)` que sume las unidades sin bucles.

Salida esperada:

```
[3.4, 1.0, 7.8]
12.2
['Café']
6
```

Sección 10 · Diseño modular

Responde en papel (razonamiento modular):

Tienes que montar un proyecto que lee pedidos de un CSV, descarta los mal formados, calcula totales por cliente y guarda un informe. Lo organizas en un paquete `tienda/`. 1. ¿Qué pondrías en un módulo `config.py` y por qué no deben estar esos valores repetidos en otros módulos? 2. ¿Por qué conviene que las funciones de `transformacion.py` sean **puras** (sin `print`, sin modificar cosas de fuera)? 3. Escribe las líneas de import que usarías **dentro** del paquete para traer una constante `IVA` de `config` y una clase `Pedido` de `modelos`. 4. Escribe la línea de import que usarías **desde** `main.py` para llamar a `ejecutar_pipeline` (que expone el `__init__.py`). 5. ¿Qué ventaja tiene esto frente a tenerlo todo en un notebook con celdas?

✍️ **Mi predicción / respuesta:**